

BOOTCAMP MYHUB · SUPPORT DE COURS

Jour 2. Pseudo-code et Python.

Support de cours à lire, annoter et garder. Imprime-le ou suis-le à l'écran. Les corrigés se font ensemble, pendant la séance.

Gratuit

Jour 2 · 2 h

Débutant bienvenu

En visio · 12 h GMT

AUJOURD'HUI TU REPARS AVEC

Une façon d'écrire un algorithme en français (le pseudo-code), puis sa traduction en Python : un petit programme qui décide.

Sujet nouveau. On ne reprend pas le travail d'un autre soir.

Prépare ça, rien de plus

- ☐ Un ordinateur (le téléphone seul ne suffit pas pour coder).
- ☐ Un dossier vide sur le Bureau, nommé `python`.
- ☐ Python installé, ou un onglet prêt : <https://www.online-python.com/>.
- ☐ Un éditeur : VS Code, Bloc-notes, ou l'éditeur du site en ligne.
- ☐ De quoi écrire : papier, ou un fichier `notes.txt`.

PAS DE PYTHON SUR LA MACHINE ?

On code dans le navigateur sur <https://www.online-python.com/>. Même exercice, zéro install. À la fin, tu copies le code dans un fichier `decision.py` pour le déposer sur MyHub.

OBJECTIF DU JOUR

Penser l'algorithme en français, puis le faire exécuter par Python. Pas besoin d'avoir déjà programmé.

Les deux parties du cours, puis le mini-projet

	ON TRAVAILLE	TU REPARS AVEC
1	Pseudo-code	Afficher, variable, concaténer, saisir, décider
2	Python : la suite	Les mêmes cinq gestes, dans le même ordre
3	Mini-projet	Ton fichier <code>decision.py</code>

LE FIL DU SOIR

Tout ce qu'on touche en pseudo-code, on le refait ensuite en Python. Ce n'est pas un autre cours : c'est la traduction.

Un algorithme, c'est une recette

Un **algorithme**, c'est une suite d'étapes claires, dans l'ordre. Le **pseudo-code**, c'est cette recette écrite en français structuré. Python viendra après, pour la même recette.

Cinq gestes ce soir, toujours dans cet ordre :

1. Afficher du texte.
2. Déclarer une variable.
3. Concaténer du texte.
4. Lire une saisie (input).
5. Faire une condition.

Chaque programme s'ouvre par `DEBUT` et se ferme par `FIN` .

Afficher du texte

AFFICHER montre un message à l'écran. Le texte va entre guillemets. L'ordre des lignes est l'ordre à l'écran.

```
DEBUT
  AFFICHER "Bonjour"
  AFFICHER "On commence."
FIN
```

À l'écran :

```
Bonjour
On commence.
```

- Guillemets = c'est du **texte**, affiché tel quel.
- Sans guillemets = c'est le **nom d'une variable** (geste 2).
- Chaque AFFICHER passe à la ligne suivante.

PIÈGE

AFFICHER Bonjour sans guillemets : la recette cherche une boîte appelée Bonjour. Le texte, lui, vit entre " .

Déclarer une variable

Une **variable**, c'est une boîte avec un nom. **Déclarer**, c'est créer la boîte et lui donner une valeur. Le signe $\leftarrow$ veut dire « mets ça dans la boîte ».

```
DEBUT
  prenom  $\leftarrow$  "Awa"
  age  $\leftarrow$  20
  AFFICHER prenom
  AFFICHER age
FIN
```

À l'écran : Awa puis 20 .

On affiche le *contenu* de la boîte, pas le mot « prenom ». "Awa" est du texte. 20 est un nombre (pas de guillemets : on pourra calculer avec).

OK	PAS OK	POURQUOI
prenom	prénom	Pas d'accent dans le nom.
age	2age	Ne commence pas par un chiffre.
envie_1	envie 1	Pas d'espace : utilise _ .

À RETENIR

Déclarer = nom + flèche + valeur. Ensuite tu réutilises le nom, sans retaper la valeur.

Concaténer du texte

Concaténer, c'est coller des morceaux de texte bout à bout, pour n'en faire qu'un. Le signe `+` colle. Pense à un espace dans les guillemets, sinon les mots se collent.

```
DEBUT
  prenom ← "Awa"
  AFFICHER "Bonjour " + prenom
  message ← "Je m'appelle " + prenom
  AFFICHER message
FIN
```

À l'écran :

```
Bonjour Awa
Je m'appelle Awa
```

- `"Bonjour " + prenom` : texte fixe + contenu de la boîte.
- Sans l'espace après Bonjour : `BonjourAwa` .
- On peut ranger le résultat dans une nouvelle variable (`message`).

PIÈGE

`+` entre deux textes = collage. `+` entre deux nombres = calcul (`20 + 1 → 21`). On ne mélange pas texte et nombre sans conversion : ça, Python le refusera.

Lire une saisie (input)

Jusqu'ici, c'est toi qui as collé « Awa » dans la recette. **LIRE**, c'est l'**input** : le programme s'arrête et attend que quelqu'un tape une réponse, puis range cette réponse dans une variable.

```
DEBUT
  AFFICHER "Ton prénom ?"
  LIRE prenom
  AFFICHER "Bonjour " + prenom
FIN
```

1. On affiche la question.
2. **LIRE prenom** déclare (ou remplit) la variable avec ce qui est tapé.
3. On affiche une seule ligne : « Bonjour » collé au prénom.

À RETENIR

LIRE crée souvent la variable en même temps. La valeur n'est plus collée dans la recette : elle arrive au moment de l'exécution.

Faire une condition

Analogie feu : vert tu avances, rouge tu t’arrêtes. SI pose une question. Une seule branche s’exécute.

```
DEBUT
  AFFICHER "Ton âge ?"
  LIRE age
  SI age >= 18 ALORS
    AFFICHER "Tu peux voter."
  SINON
    AFFICHER "Pas encore."
  FIN SI
FIN
```

MOT	RÔLE
SI ... ALORS	Si c’est vrai, fais ce bloc.
SINON	Sinon, fais l’autre bloc.
FIN SI	La décision est terminée.
>= < =	Comparer deux valeurs.

Parcours : si on tape 16 → « Pas encore. » Si on tape 20 → « Tu peux voter. »

À BIEN VERROUILLER

Ce qui est en retrait sous le SI appartient au « si ». Sans FIN SI , on ne sait plus où la recette reprend.

Écris deux programmes légers

Au crayon ou dans `notes.txt`. Deux petits programmes qui utilisent les cinq gestes. Pas de Python encore.

TU AS LE DROIT D'UTILISER

DEBUT · FIN · AFFICHER · LIRE · ← · + · SI ... ALORS · SINON · FIN SI .

PROGRAMME 1 · 6 MIN · ACCUEIL

Écris un programme qui demande le prénom, puis affiche **une seule ligne** « Bonjour » collé au prénom, avec + .

PROGRAMME 2 · 6 MIN · ADMIS OU PAS

Écris un programme qui demande une note sur 20. Si la note est supérieure ou égale à 10, afficher « Admis ». Sinon, afficher « Ajourné ».

Même cinq gestes, dans le même ordre

Cette partie est **réservée à Python**. Ce n'est pas un autre sujet : c'est la suite. On reprend exactement ce qu'on a touché en pseudo-code.

EN PSEUDO-CODE	EN PYTHON
Afficher du texte · AFFICHER	print
Déclarer une variable · ←	=
Concaténer · +	+ (textes uniquement)
Lire une saisie · LIRE	input
Condition · SI / SINON / FIN SI	if / else + le retrait

TU NE DEVINES PAS

Tu traduis. Chaque ligne de la recette a une ligne Python.

Afficher du texte · print

Même exemple qu'en pseudo-code.

Pseudo-code

```
DEBUT
  AFFICHER "Bonjour"
  AFFICHER "On commence."
FIN
```

Python

```
print("Bonjour")
print("On commence.")
```

- Les guillemets entourent un **texte**.
- Sans guillemets, Python cherche une **variable**.
- `print(Bonjour)` → erreur `NameError` .

Déclarer une variable · =

En maths, = veut dire « égal ». En Python, = déclare la boîte et range la valeur. C'est le ← du pseudo-code.

Pseudo-code

```
prenom ← "Awa"  
age ← 20  
AFFICHER prenom  
AFFICHER age
```

Python

```
prenom = "Awa"  
age = 20  
print(prenom)  
print(age)
```

"20" est du texte. 20 est un nombre. "20" + "1" donne "201". 20 + 1 donne 21.

Concaténer du texte · +

Même collage qu'en pseudo-code. + entre deux textes.

Pseudo-code

```
prenom ← "Awa"  
AFFICHER "Bonjour " + prenom  
message ← "Je m'appelle " + prenom  
AFFICHER message
```

Python

```
prenom = "Awa"  
print("Bonjour " + prenom)  
message = "Je m'appelle " + prenom  
print(message)
```

NOMBRE + TEXTE = ERREUR

"Tu as " + 20 plante. Il faut transformer le nombre : `print("Tu as " + str(20) + " ans")`.
`str(...)` = « rends-moi ça en texte ».

Lire une saisie · input

Même accueil. Cette fois, une seule ligne grâce au + .

Pseudo-code

```
AFFICHER "Ton prénom ?"  
LIRE prenom  
AFFICHER "Bonjour " + prenom
```

Python

```
prenom = input("Ton prénom ? ")  
print("Bonjour " + prenom)
```

À RETENIR

`input` rend **toujours un texte**. Pour une note ou un âge, on convertit : `age = int(input("Ton âge ? "))` .

Faire une condition • if / else

Même décision d'âge. Plus de `FIN SI` : le retrait ferme le bloc.

Pseudo-code

```
LIRE age
SI age >= 18 ALORS
    AFFICHER "Tu peux voter."
SINON
    AFFICHER "Pas encore."
FIN SI
```

Python

```
age = int(input("Ton âge ? "))
if age >= 18:
    print("Tu peux voter.")
else:
    print("Pas encore.")
```

Les deux-points `:` ouvrent le bloc. 4 espaces = ce qui appartient au `if` ou au `else`. `= range`. `==` pose la question « est-ce égal ? ».

ERREUR	RÉFLEXE
<code>SyntaxError</code>	Parenthèse, deux-points ou guillemet manquant.
<code>NameError</code>	Variable oubliée, ou texte sans guillemets.
<code>IndentationError</code>	Le retrait sous le <code>if</code> manque.
<code>ValueError</code>	<code>int("Awa")</code> : ce n'est pas un nombre.

Suite : les deux mêmes programmes, en Python

Ceux que tu as écrits en pseudo-code. Tu les traduis, tu ne recommences pas un autre sujet.

PROGRAMME 1 · EN PYTHON · ACCUEIL

Traduis ta recette : demander le prénom, afficher **une seule ligne** avec `+` : « Bonjour » collé au prénom.

PROGRAMME 2 · EN PYTHON · ADMIS OU PAS

Traduis ta recette : demander une note sur 20. Si elle est ≥ 10 , afficher « Admis ». Sinon, « Ajourné ». Pense à `int(...)` pour la note.

Assez pour payer ?

Un problème du quotidien. D'abord la recette, ensuite le code. Fichier : `python/decision.py` .

Le problème

On demande le **prix** d'un article, puis la **somme** que tu as. Si tu as assez, le programme affiche le reste. Sinon, il affiche ce qui manque.

Étape A · le pseudo-code (non négociable)

Écris-le en commentaires en haut du fichier, avec `#` .

```
# DEBUT
#   AFFICHER "Prix de l'article ?"
#   LIRE prix
#   AFFICHER "Tu as combien ?"
#   LIRE argent
#   SI argent >= prix ALORS
#     reste ← argent - prix
#     AFFICHER "C'est bon. Il te reste : " + reste
#   SINON
#     manque ← prix - argent
#     AFFICHER "Pas assez. Il manque : " + manque
#   FIN SI
# FIN
```

Étape B · la traduction Python

```
prix = int(input("Prix de l'article ? "))
argent = int(input("Tu as combien ? "))

if argent >= prix:
    reste = argent - prix
    print("C'est bon. Il te reste : " + str(reste))
else:
    manque = prix - argent
    print("Pas assez. Il manque : " + str(manque))
```

Cahier des charges

- ☐ Le fichier s'appelle `decision.py` .
- ☐ Le pseudo-code est en commentaires en haut du fichier.
- ☐ Deux `input` convertis avec `int` : `prix` et `argent`.
- ☐ Un `if` / `else` : assez, ou pas assez.
- ☐ Tu l'as lancé au moins deux fois : une fois « assez », une fois « pas assez ».

EXEMPLE POUR TESTER

Prix 1500, argent 2000 → reste 500. Prix 1500, argent 800 → il manque 700.

Les cinq blocages classiques

SYMPTÔME	RÉFLEXE
Le fichier s'appelle <code>decision.py.txt</code>	Affiche les extensions dans l'explorateur Windows.
« python n'est pas reconnu »	Passe sur online-python.com .
Le curseur clignote, rien ne s'affiche	Le programme attend <code>input</code> . Tape un nombre, Entrée.
<code>ValueError</code>	Tape un nombre, pas un mot. Pas d'espace bizarre.
<code>IndentationError</code>	Tout ce qui est sous <code>if</code> / <code>else</code> est en retrait.

Comment lancer le fichier (si Python est installé)

1. Ouvre un terminal dans le dossier `python` .
2. Tape `python decision.py` (parfois `py decision.py` sous Windows).
3. Réponds aux deux questions, Entrée après chaque ligne.

MINI-EXO DE FIN DE SÉANCE

Dépose `decision.py` sur MyHub, espace étudiant, devoir du jour 2. Tu peux coller le code si le fichier refuse de partir.

Inscription : prénom, nom, téléphone. Connexion : prénom + téléphone.

Glossaire du jour

MOT	EN UNE PHRASE
Algorithme	Une suite d'étapes pour résoudre un problème.
Pseudo-code	L'algorithme écrit en français structuré.
Programme	La même recette, dans un langage exécutable.
Python	Le langage de ce soir.
Variable	Une boîte nommée qui range une valeur.
<code>AFFICHER</code> / <code>print</code>	Montrer un message.
<code>LIRE</code> / <code>input</code>	Demander une réponse.
<code>←</code> / <code>=</code>	Ranger dans une variable.
<code>==</code>	Comparer : est-ce égal ?
<code>SI</code> / <code>if</code>	Choisir une branche si c'est vrai.
Indentation	Le retrait qui marque le bloc sous un <code>if</code> .
<code>int</code>	Un nombre entier, pour calculer.
Concaténation	Coller des textes avec <code>+</code> .
<code>str</code>	Transformer un nombre en texte, pour pouvoir le coller.

Checklist avant d'envoyer ton fichier

- ☐ Le fichier s'appelle `decision.py`.
- ☐ Le pseudo-code est en commentaires (`#`) en haut.
- ☐ Le Python en dessous traduit bien cette recette.
- ☐ Deux tests : assez d'argent, et pas assez.
- ☐ Tu déposes sur MyHub, pas sur WhatsApp.

POUR LE JOUR 3 · DATA

Nouveau sujet encore. Demain, un invité. Même heure, même visio, ordinateur encore obligatoire.